DEVOPS CAPSTONE

MediDesk

From source code to a monitored Kubernetes deployment

APPLICATION	DELIVERY
React / Vite • FastAPI • PostgreSQL	GitHub Actions • Docker • Helm
PLATFORM	**OBSERVABILITY**
Terraform • AWS EKS • Kubernetes	Prometheus • Grafana

Technical presentation • Architecture • Delivery • Observability

What MediDesk demonstrates

A full delivery path: application code, repeatable infrastructure, deployment, and operational visibility.

01 Build

A frontend and API packaged as deployable application components.

02 Deploy

Terraform-provisioned infrastructure and Kubernetes workloads released with Helm.

03 Observe

Prometheus scrapes application metrics and Grafana turns them into dashboards.

Engineering goals

- Reproducibility — describe infrastructure and workloads as code.
- Automation — validate changes and build container images through CI.
- Operability — expose health endpoints, ingress access, and metrics.

Architecture at a glance

The request path and the delivery/observability paths are separate but connected.

REQUEST PATH

User / Browser → Ingress → React frontend → FastAPI backend → PostgreSQL

DELIVERY PATH

GitHub Actions → tests / build / scans → container registry → Kubernetes release

INFRASTRUCTURE PATH

Terraform → AWS VPC / EKS and supporting cloud resources

OBSERVABILITY PATH

Backend metrics → Prometheus → Grafana dashboards

Validate the exact deployed state during the recording; architecture boxes describe the intended system, not proof by themselves.

The stack and the job each tool does

Tools grouped by responsibility.

Application

React + Vite frontend
FastAPI backend
PostgreSQL data store

Packaging

Docker images
Docker Compose for local development

Infrastructure

Terraform
AWS VPC + EKS
IAM / networking / node capacity

Deployment

Kubernetes
Helm chart and values
Ingress routing

Observability

Prometheus
ServiceMonitor
Grafana dashboards

Automation

GitHub Actions
Tests, frontend build, image scanning/publishing

Where to look in the repository

Show the repository tree and open one representative file per area.

backend/

API implementation, health endpoint, metrics endpoint, tests and dependencies

frontend/

React/Vite UI, package manifest, production build configuration

terraform/

AWS infrastructure definitions, variables and outputs

helm/

Kubernetes templates and configurable application values

.github/workflows/

CI pipeline definition and automated checks

demo/

Presentation assets, evidence screenshots and walkthrough materials

Confirm these paths match the current repository before recording; do not invent filenames while narrating.

A code change becomes a validated image

The workflow file is the source of truth; show a real successful run in GitHub Actions.

01 Push

Commit reaches GitHub

02 Test

Backend pytest + frontend build

03 Scan

Trivy checks for HIGH / CRITICAL findings

04 Publish

Docker images tagged with commit SHA to GHCR

05 Deploy

EKS deployment is conditional on configured enablement

Recording proof

- Trigger a small, safe commit → open the run → show each job and the final green result.
- Only describe steps as successful when the actual workflow run confirms them.

Terraform makes the cloud setup reviewable

Show plan output and the AWS Console; redact account-specific or sensitive values if visible.

Plan before apply

terraform fmt -check
terraform validate
terraform plan

Cloud foundation

VPC and networking
EKS control plane
Worker nodes and IAM

Why it matters

Version-controlled infrastructure
Repeatable review
Reduced manual drift

Live walkthrough checklist

- Run terraform plan and show whether changes are pending.
- Open AWS Console and point out the VPC and EKS cluster.
- Do not run apply or destroy during recording unless a change is intentional, reviewed, and approved.

Argo CD: the required proof point

GitOps means the desired state in Git is reconciled with the running cluster.

Desired state

Manifests / Helm configuration in the repository

Argo CD

Watches the source and compares desired vs live state

Cluster state

Application health and sync status reflect deployed resources

Required on-screen evidence

- Open the Argo CD application and show Synced + Healthy.
- If Argo CD is not installed/configured yet, treat this as an outstanding requirement—not a completed feature.
- Do not claim GitOps reconciliation unless the live application page proves it.

Kubernetes health and traffic routing

Show actual cluster output, then follow the request through the ingress endpoint.

Workloads

kubectl get pods -n medidesk -o wide

Services

kubectl get svc -n medidesk

Ingress

kubectl get ingress -n medidesk

Release

helm list -n medidesk

Health check

- Show all expected pods Ready / Running and verify the application health endpoint returns HTTP 200.
- Use the correct namespace and current output; this slide is a guide, not a substitute for live proof.

Metrics become operational signals

Prove the app is being scraped, then show a dashboard using real time-series data.

Instrument

Backend exposes /api/metrics

Collect

Prometheus target status is UP

Explore

Grafana panels query Prometheus metrics

Prometheus proof

Targets page → backend target → State: UP

Grafana proof

Open dashboard → set recent time range → show populated panels

Use a live dashboard with non-empty data. A blank panel is not proof of observability.

What to demonstrate—and what not to fake

A strong submission distinguishes implementation evidence from planned or unverified features.

Confirmed in current project context

EKS cluster and worker nodes
Helm release and healthy app endpoints
Prometheus target UP
Grafana installed and enabled

Must be verified live before recording

End-to-end CI run after a fresh push
Terraform plan output and AWS Console
Argo CD app showing Synced + Healthy
Grafana panels populated with current metrics

Security hygiene

- Avoid exposing tokens, passwords, kubeconfig contents, private keys, or unnecessary account identifiers in the recording.
- If a required component is missing, fix it or clearly disclose it before submission.

TAKEAWAYS

MediDesk is more than an app.

It is a delivery path with infrastructure, deployment, and operational visibility.

Repeatable	Automated	Observable
Terraform + Helm	GitHub Actions	Prometheus + Grafana

The live walkthrough supplies the evidence; the slides explain the system and engineering decisions.